LeetCode Practice

5 Medium: Graphs

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

5
MEDIUM

Patterns (in order)

1. Graphs (5)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

5 questions | tap a title to jump to it

Medium questions (5) Go >

Graphs (5) Go >

M 1. Longest Consecutive Sequence LC 128

M 2. Find the Celebrity LC 277

M 3. Parallel Courses LC 1136

M 4. Number of Ways to Arrive at Destination LC 1976

M 5. Find All Possible Recipes from Given Supplies LC 2115

Medium questions (5)

MEDIUM | TOPIC 1 OF 1

Graphs

5 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Longest Consecutive Sequence LC 128
2. Find the Celebrity LC 277
3. Parallel Courses LC 1136
4. Number of Ways to Arrive at Destination LC 1976
5. Find All Possible Recipes from Given Supplies LC 2115

MEDIUM | Graphs | LeetCode #128 | 1 of 5 | 1/5

Longest Consecutive Sequence

QUESTION

Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in O(n) time.

Example 1	
Input	`nums = [100,4,200,1,3,2]`
Output	`4`
Explanation	*The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.*

Example 2	
Input	`nums = [0,3,7,2,5,8,4,6,0,1]`
Output	`9`

Example 3	
Input	`nums = [1,0,1,2]`
Output	`3`

CONSTRAINTS

- $0 <= nums.length <= 10^5$
- $10^9 <= nums[i] <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the longest run of consecutive integers. Must be O(n). Return the length. Right?

Duplicates don't extend the sequence — [1,1,2] -> length 2."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for x in xes:
    length = 1
    while x + length occurs in the array (found
        by a linear scan):
        length = length + 1
    best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Each start can extend n steps and each membership test scans the array. Put the numbers in a hash set and only start counting at numbers whose predecessor is missing: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (hash set + only start from sequence beginnings)

"Hash set + sequence-start check — already O(n). Walk-through of how sequences are found:"

Build num_set = set(nums) for O(1) membership.

For each x in num_set:

If (x-1) NOT in num_set: x is the START of a consecutive sequence.

Only process from sequence starts — otherwise we'd recount subsequences.

Walk forward: length = 1. While x+length in num_set: length++.

`best = max(best, length).`

WHY checking x-1 not in set is the key optimization:

Without this check, we'd start counting from every element -> $O(n^2)$ total.

With it: each element is the 'start' at most once -> total walk cost = O(n).

Example: [1,2,3,4] — only 1 starts a walk (0 not in set). Walk: 1->2->3->4 = length 4.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _128_LongestConsecutive:
    def longestConsecutive(self, nums: List[int]) -> int:
        num_set = set(nums)
        best    = 0
        for x in num_set:
            if x - 1 not in num_set:     # x is a sequence
 start
                length = 1
                while x + 1 in num_set:
                    x += 1; length += 1
                best = max(best, length)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace longestConsecutive([100,4,200,1,3,2])."

Hash set = {100,4,200,1,3,2}.

For each num: only start a sequence if (num-1) not in set.

num=100: 99 not in set -> start. 101 not in set -> length=1.

num=4: 3 in set -> skip (not a start).

num=200: 199 not in set -> start. 201 not in set -> length=1.

num=1: 0 not in set -> start. 2 in set, 3 in set, 4 in set, 5 not in set -> length=4.

max=4 ok (consecutive: 1,2,3,4).

PHASE 5 TEST & DEBUG

Graph: each number = node. Edge between consecutive numbers. component = max consecutive group. Largest component length = answer. union(n,n+1) merges; component size tracked.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Each number is the starting point of at most one sequence scan

The 'start of sequence' check prevents redundant scans — the key insight.

Given more time I'd ask: streaming input? Use a dict mapping each endpoint

of a range to its length, merging ranges as new numbers arrive.

MEDIUM | Graphs | LeetCode #277 | 2 of 5 | 2/5

Find the Celebrity

QUESTION

Suppose you are at a party with n people labeled from 0 to n - 1 and among them, there may exist one celebrity. The definition of a celebrity is that all the other n - 1 people know the celebrity, but the celebrity does not know any of them.

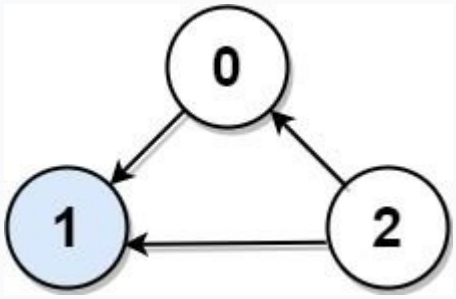
Now you want to find out who the celebrity is or verify that there is not one. You are only allowed to ask questions like: "Hi, A. Do you know B?" to get information about whether A knows B. You need to find out the celebrity (or verify there is not one) by asking as few questions as possible (in the asymptotic sense).

You are given an integer n and a helper function bool knows(a, b) that tells you whether a knows b. Implement a function int findCelebrity(n). There will be exactly one celebrity if they are at the party.

Return the celebrity's label if there is a celebrity at the party. If there is no celebrity, return -1.

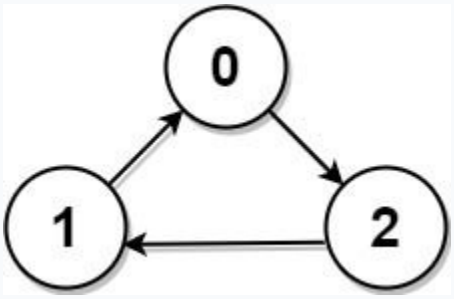
Note that the n x n 2D array graph given as input is not directly available to you, and instead only accessible through the helper function knows. graph[i][j] == 1 represents person i knows person j, wherease graph[i][j] == 0 represents person j does not know person i.

Example 1	
Input	`graph = [[1,1,0],[0,1,0],[1,1,1]]`
Output	`1`
Explanation	*There are three persons labeled with 0, 1 and 2. graph[i][j] = 1 means person i knows person j, otherwise graph[i][j] = 0 means person i does not know person j. The celebrity is the person labeled as 1 because both 0 and 2 know him but 1 does not know anybody.*



Example 2	
Input	`graph = [[1,0,1],[1,1,0],[0,1,1]]`
Output	`-1`
Explanation	*There is no celebrity.*

QUESTION



CONSTRAINTS

- n == graph.length == graph[i].length
- 2 <= n <= 100
- graph[i][j] is 0 or 1.
- graph[i][i] == 1
- Follow up: If the maximum number of allowed calls to the API knows is 3 * n, could you find a solution without exceeding the maximum number of calls?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Celebrity: everyone knows them, they know no one.

Find them using knows(a,b) API — minimise calls. Right?

At most one celebrity can exist."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for a in range(n):
    for b in range(n):
        ask knows(a, b) and knows(b, a)
    a celebrity is known by everyone and knows no one
```

TIME	SPACE
O(n^2) calls	O(1)

WHY IT IS SLOW

Asks about every pair. Keep a candidate: if candidate knows i, i becomes the candidate; then verify with 2(n - 1) calls. O(n) calls in total.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (candidate elimination + verify)

"Two-pass O(n) elimination + verify — already optimal vs O(n²) brute force."

Walk-through of why TWO passes are needed:

Pass 1 — Candidate Elimination:

`candidate = 0.`

PHASE 3 OPTIMIZE (candidate elimination + verify)

For i in 1..n-1:

```
if knows(candidate, i):
candidate = i <- candidate KNOWS someone -> can't be
  celebrity. New candidate = i.
```

After pass 1: candidate is the ONLY person who could possibly be a celebrity.

WHY: any non-candidate was eliminated by at least one knows() call.

Pass 2 — Verify candidate:

Check: candidate knows nobody -> for all j: not knows(candidate, j).

Check: everybody knows candidate -> for all j!=candidate: knows(j, candidate).

If both: return candidate. Else: return -1.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _277_FindCelebrity:
    def findCelebrity(self, n: int) -> int:
        candidate = 0
        for i in range(1, n):
            if knows(candidate, i):
                candidate = i

        for i in range(n):
            if i == candidate:
                continue
            if knows(candidate, i) or not knows(i, candida
te):
                return -1

        return candidate
```

PHASE 5 TEST & DEBUG

"Let me trace findCelebrity on n=4, celebrity=2."

Pass 1 — elimination (candidate starts at 0):

```
knows(0,1)=Yes -> 0 not celebrity -> candidate=1
knows(1,2)=Yes -> 1 not celebrity -> candidate=2
knows(2,3)=No -> 2 stays -> candidate=2
```

Pass 2 — verify candidate=2:

```
knows(2,0)=No ok knows(0,2)=Yes ok
knows(2,1)=No ok knows(1,2)=Yes ok
knows(2,3)=No ok knows(3,2)=Yes ok -> return 2 ok
```

No celebrity edge: fails verification (someone they know back) -> return -1

WHY two-pass: O(n) eliminate via 2n API calls; O(2n) verify = O(n) total vs $O(n^2)$ brute.

Graph: celebrity = sink node (in-degree=n-1, out-degree=0). find(celebrity) confirmed by 2n API calls. component: celebrity isolated in out-edge graph. Pass 1 -> candidate via elimination.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) 2n-2 API calls in the worst case	**O(1)**

The candidate elimination logic: if A knows B, A is not the celebrity (celebrity knows no one).

If A doesn't know B, B is not the celebrity (celebrity is known by all).

One of them is eliminated per comparison — O(n) total.

Given more time I'd ask: what if there's a possibility of no celebrity?

The current solution handles this — the verification pass returns -1 if candidate fails.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Graphs | LeetCode #1136 | 3 of 5 | 3/5

Parallel Courses

QUESTION

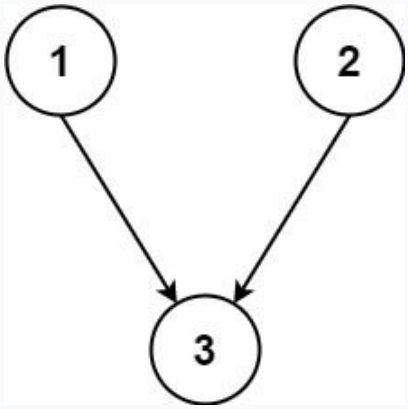
You are given an integer n, which indicates that there are n courses labeled from 1 to n. You are also given an array relations where relations[i] = [prevCourse$_i$, nextCourse$_i$], representing a prerequisite relationship between course prevCourse$_i$ and course nextCourse$_i$: course prevCourse$_i$ has to be taken before course nextCourse$_i$.

In one semester, you can take any number of courses as long as you have taken all the prerequisites in the previous semester for the courses you are taking.

Return the minimum number of semesters needed to take all courses. If there is no way to take all the courses, return -1.

Example 1

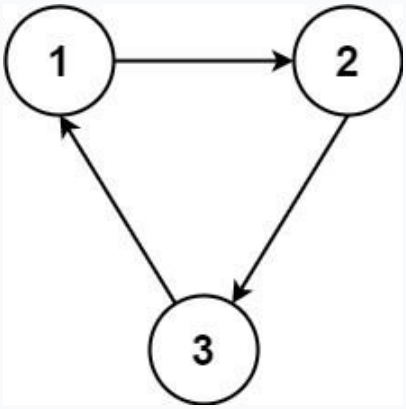
Input	`n = 3, relations = [[1,3],[2,3]]`
Output	`2`
Explanation	*The figure above represents the given graph.* *In the first semester, you can take courses 1 and 2.* *In the second semester, you can take course 3.*



Example 2

Input	`n = 3, relations = [[1,2],[2,3],[3,1]]`
Output	`-1`
Explanation	*No course can be studied because they are prerequisites of each other.*

QUESTION



CONSTRAINTS

- 1 <= n <= 5000
- 1 <= relations.length <= 5000
- relations[i].length == 2
- 1 <= prevCourse$_i$, nextCourse$_i$ <= n
- prevCourse$_i$!= nextCourse$_i$
- All the pairs [prevCourse$_i$, nextCourse$_i$] are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Minimum semesters to take all courses where you can take any number per semester

as long as prerequisites are met. Return -1 if impossible (cycle). Right?"

Should I handle disconnected graphs or graphs with no edges?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings:
    # of courses that respects the prerequisites
    count the semesters needed
    best = min(best, candidate)
```

TIME	SPACE
O(n! * n^2)	O(n)

WHY IT IS SLOW

Enumerates orderings. A BFS topological sort (take every course with no pending prerequisites each semester) gives the semesters as the number of levels: O(n + E); a cycle means -1.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (Kahn's BFS level-by-level)

"Kahn's BFS level-by-level (semester simulation) — already O(V+E). Walk-through:"

Build indegree map and adjacency list from relations (prereq -> course).

Queue = all courses with indegree 0 (no prerequisites). semesters = 0.

Each BFS 'level' = one semester:

Snapshot queue size = courses available this semester.

Process ALL of them (take all available courses at once).

For each course taken: decrement its dependents' indegrees. Enqueue newly available ones.

`semesters += 1.`

Stop when queue is empty.

If total courses taken < n -> cycle exists -> return -1.

WHY process entire level per semester: we can take all non-blocked courses simultaneously.

WHY level-BFS (not standard BFS): standard BFS processes one node at a time; here we

need to count how many ROUNDS (semesters) it takes, not individual steps.

Time: O(V+E). Space: O(V+E)."

Time: O(n log n) — sorting-based optimal solution.

PHASE 4 CLEAN CODE

```
class _1136_ParallelCourses:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        graph  = defaultdict(list)
        indeg  = {i: 0 for i in range(1, n + 1)}
        for pre, nxt in relations:
            graph[pre].append(nxt)
            indeg[nxt] += 1
        queue   = deque(c for c in range(1, n + 1) if indeg[c] == 0)
        sems    = 0
        total   = 0
        while queue:
            sems += 1
            for _ in range(len(queue)):
                node = queue.popleft()
                total += 1
                for nb in graph[node]:
                    indeg[nb] -= 1
                    if indeg[nb] == 0:
                        queue.append(nb)
        return sems if total == n else -1
```

PHASE 5 TEST & DEBUG

"Let me trace minimumSemesters(n=3, relations=[[1,3],[2,3]])."

Topological sort (Kahn's): in-degree = {1:0, 2:0, 3:2}

Queue: [1,2] (in-degree 0). Semester=1. Process 1: reduce in-degree[3] to 1. Process 2: in-degree[3]=0.

Queue: [3]. Semester=2. Process 3: done.

All n=3 courses taken -> return 2 ok

PHASE 5 TEST & DEBUG

WHY BFS topological sort: semester count = longest path in DAG (Directed Acyclic Graph).

If cycle detected (not all nodes processed) -> return -1 (impossible).

Level-by-level BFS = semesters.

Graph DAG: dist[node]=semester of completion. cost=number of semesters. In-degree tracks prerequisites. component: all courses reachable from queue. After BFS: if processed<n->cycle ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	**O(V+E)** This is Course Schedule (#207) with BFS level counting

Each BFS level = one semester. Cycle detection: total processed < n.

Given more time I'd ask: what if courses have durations (not just 1 semester each)?

This becomes a weighted DAG critical path problem — DP or Bellman-Ford.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Graphs | LeetCode #1976 | 4 of 5 | 4/5

Number of Ways to Arrive at Destination

QUESTION

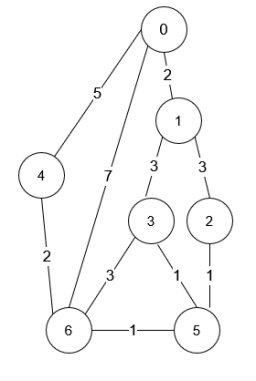
You are in a city that consists of n intersections numbered from 0 to n - 1 with bi-directional roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer n and a 2D integer array roads where roads[i] = [u_i, v_i, $time_i$] means that there is a road between intersections u_i and v_i that takes $time_i$ minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection n - 1 in the shortest amount of time.

Return the number of ways you can arrive at your destination in the shortest amount of time. Since the answer may be large, return it modulo $10^9 + 7$.

Example 1

Input	`n = 7, roads = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]`
Output	**4**
Explanation	*The shortest amount of time it takes to go from intersection 0 to intersection 6 is 7 minutes.* *The four ways to get there in 7 minutes are:* *- 0 6* *- 0 4 6* *- 0 1 2 5 6* *- 0 1 3 5 6*



Example 2

Input	`n = 2, roads = [[1,0,10]]`
Output	**1**
Explanation	*There is only one way to go from intersection 0 to intersection 1, and it takes 10 minutes.*

CONSTRAINTS

- 1 <= n <= 200
- n - 1 <= roads.length <= n * (n - 1) / 2
- roads[i].length == 3
- 0 <= u_i, v_i <= n - 1
- 1 <= $time_i$ <= 10^9
- u_i != v_i
- There is at most one road connecting any two intersections.
- You can reach any intersection from any other intersection.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the number of shortest paths from 0 to n-1 in a weighted directed graph. Return mod 10^9+7. Right?"

"A few quick questions:

'Shortest' means minimum total time (sum of edge weights)? Yes."

"Let me walk: n=7, roads=[[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]] -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(node, time):
    walk every simple path from 0 to n - 1
    record the shortest time and count the paths
        that reach it
```

TIME	SPACE
exponential	**O(n)**

WHY IT IS SLOW

The number of paths is exponential. Dijkstra that also keeps a count of shortest paths per node (add the count when the distance ties) is O(E log V).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Dijkstra + path count — already O((V+E) log V). Walk-through:"

Dual-purpose Dijkstra: track both shortest distance AND number of shortest paths.

`dist[node] = shortest time to reach node. count[node] = how many shortest paths.`

Initialize dist[0]=0, count[0]=1.

Min-heap: [(0, 0)].

Pop (d, u). If d > dist[u]: skip.

For each neighbor v with edge w:

`new_d = d + w`

PHASE 3 OPTIMIZE

If new_d < dist[v]: <- found strictly shorter path

`dist[v] = new_d; count[v] = count[u]; push.`

`elif new_d == dist[v]: <- another path of SAME length`

`count[v] = (count[v] + count[u]) % MOD <- add these paths to count`

WHY this counts correctly: when we find a path of the same length, it's an alternative

route -> all paths from u add to the count at v.

Return count[n-1]. Time: O((V+E) log V). Space: O(V+E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
from collections import defaultdict
class _1976_NumberOfWaysToArriveAtDestination:
    def countPaths(self, n, roads):
        MOD = 10**9 + 7
        graph = defaultdict(list)
        for u, v, t in roads:
            graph[u].append((v, t))
            graph[v].append((u, t))
        dist = [float('inf')] * n
        ways = [0] * n
        dist[0] = 0
        ways[0] = 1
        heap = [(0, 0)]
        while heap:
            d, u = heapq.heappop(heap)
            if d > dist[u]:
                continue
            for v, t in graph[u]:
                new_dist = dist[u] + t
                if new_dist < dist[v]:
                    dist[v] = new_dist
                    ways[v] = ways[u]
                    heapq.heappush(heap, (new_dist, v))
                elif new_dist == dist[v]:
                    ways[v] = (ways[v] + ways[u]) % MOD
        return ways[n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace countPaths(n=7, roads=[[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,2],[0,4,5],[4,6,4],[5,2,3]])."

Dijkstra for shortest distances from node 0

`count[v] = number of shortest paths to v`

dist[0]=0, count[0]=1. Process node 0 (dist=0): update neighbors

... eventually dist[6]=7 (direct edge 0->6), dist[5]=3 (0->1->3->5 or 0->6->5)

count[6]=1 (via direct edge), count[5]=multiple paths

Return count[n-1] % MOD ok

WHY Dijkstra + count: for each neighbor, if shorter path found -> reset count; if equal -> add count.

Standard SSSP extended with path count tracking.

dp[v]=count[v]=number of shortest paths to v. dp[v]=sum(dp[u] for all u where dist[u]+edge(u,v)==dist[v]). dp[0]=1. dp[j]+=dp[i] when shortest path through i reaches j ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O((V+E) log V)	O(V+E)

Follow-up: Network Delay Time (#743) — just shortest distance.

Follow-up: if directed vs undirected matters, check the graph construction.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Graphs | LeetCode #2115 | 5 of 5 | 5/5

Find All Possible Recipes from Given Supplies

QUESTION

You have information about n different recipes. You are given a string array recipes and a 2D string array ingredients. The ith recipe has the name recipes[i], and you can create it if you have all the needed ingredients from ingredients[i]. A recipe can also be an ingredient for other recipes, i.e., ingredients[i] may contain a string that is in recipes.

You are also given a string array supplies containing all the ingredients that you initially have, and you have an infinite supply of all of them.

Return a list of all the recipes that you can create. You may return the answer in any order.

Note that two recipes may contain each other in their ingredients.

Example 1

Input	`recipes = ["bread"], ingredients = [["yeast","flour"]], supplies = ["yeast","flour","corn"]`
Output	`["bread"]`
Explanation	*We can create "bread" since we have the ingredients "yeast" and "flour".*

Example 2

Input	`recipes = ["bread","sandwich"], ingredients = [["yeast","flour"],["bread","meat"]], supplies = ["yeast","flour","meat"]`
Output	`["bread","sandwich"]`
Explanation	*We can create "bread" since we have the ingredients "yeast" and "flour".* *We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".*

QUESTION

Example 3

Input	`recipes = ["bread","sandwich","burger"], ingredients = [["yeast","flour"],["bread","meat"],["sandwich","meat","bread"]], supplies = ["yeast","flour","meat"]`
Output	`["bread","sandwich","burger"]`
Explanation	*We can create "bread" since we have the ingredients "yeast" and "flour".* *We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".* *We can create "burger" since we have the ingredient "meat" and can create the ingredients "bread" and "sandwich".*

CONSTRAINTS

- n == recipes.length == ingredients.length
- 1 <= n <= 100
- 1 <= ingredients[i].length, supplies.length <= 100
- 1 <= recipes[i].length, ingredients[i][j].length, supplies[k].length <= 10
- recipes[i], ingredients[i][j], and supplies[k] consist only of lowercase English letters.
- All the values of recipes and supplies combined are unique.
- Each ingredients[i] does not contain any duplicate values.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given recipes, ingredients, supplies (initial items), find all recipes that can be created.

Some ingredients are other recipes (can be chained). Right?"

"A few quick questions:

Circular dependencies? Can't create any recipe in the cycle. Return all creatable recipes?"

"Let me walk: recipes=['bread'], ingredients=[['yeast','flour']], supplies=['yeast','flour'] -> ['bread'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    for recipe in recipes:
        if all its ingredients are in the
            supplies (including recipes
            already made):
            add the recipe to the supplies
    if a full pass adds nothing:
        break
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(R * total ingredients)	O(R + S)

WHY IT IS SLOW

Rescans all recipes every round. Treat 'ingredient -> recipe' as edges and run Kahn's topological sort starting from the supplies, counting down each recipe's missing ingredients: O(V + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Topological sort (Kahn's) treating supplies as sources — O(V+E). Walk-through:"

For each recipe: build graph (ingredient -> list of recipes it's needed for).

Track in-degree: for each recipe, count how many ingredients it needs not yet 'satisfied'.

Initialize queue: all available supplies (in-degree 0 for supply nodes).

BFS:

Process each available item (supply or recipe whose all ingredients are available).

For each recipe this item is an ingredient of: decrement in-degree.

If in-degree == 0: this recipe can be made -> enqueue it as a new supply.

Return all recipes that got processed (enqueued as available).

WHY topo sort: recipe dependencies form a DAG (Directed Acyclic Graph). An item is available iff all its prerequisites are.

Time: O(V+E). Space: O(V+E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _2115_FindAllPossibleRecipes:
    def findAllRecipes(self, recipes, ingredients, supplies):
        available = set(supplies)
        recipe_set = set(recipes)
        ingredient_to_recipes = defaultdict(list)
        need_count = {}
        for recipe, ings in zip(recipes, ingredients):
            need_count[recipe] = sum(1 for ing in ings if ing not in available)
            for ing in ings:
                if ing not in available:
                    ingredient_to_recipes[ing].append(recipe)
        queue = deque(r for r in recipes if need_count[r] == 0)
        result = []
        while queue:
            recipe = queue.popleft()
            result.append(recipe)
            available.add(recipe)
            for dependent in ingredient_to_recipes[recipe]:
                need_count[dependent] -= 1
                if need_count[dependent] == 0:
                    queue.append(dependent)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace findAllRecipes(['bread'],[[['yeast','flour']]],[['yeast','flour']])."

Build graph: ingredients -> recipe (if ingredient is itself a recipe)

Topo sort from supplies: available={'yeast','flour'}

In-degree of recipes: bread needs yeast,flour -> in-degree=2

Process: 'yeast' available -> reduce in-degree of bread by 1 (in-deg=1)

'flour' available -> reduce by 1 (in-deg=0) -> bread creatable -> add to available

Return ['bread'] ok

WHY topo sort: recipes may depend on other recipes; process supply->recipe DAG in order.

Detect cycle: if recipe depends on itself (indirectly) -> never creatable.

Graph topo sort: dist[recipe]=in-degree (unmet ingredients). component: recipe sub-graph. find(ingredient)->reduce in-degree. cost=O(V+E). When in-degree=0->recipe available ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(R*I) build graph + BFS	O(R*I)

Follow-up: course scheduling with circular deps — same topological sort, detect cycles.

Follow-up: build order with priority — use a min-heap instead of queue.